Working With Text.

Your apps can already store data. Today they learn to READ — text out of PDFs, patterns out of strings, names and emails out of resumes. The foundation for the AI exercises later this week.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

11 of 25

AGENDA · DAY 11

How we'll spend our 90 minutes today.

Regex, PDFs, Word docs, and a resume contact extractor. The text-processing skills behind 4 of the 10 group project ideas.

DURATION
90 min

00:00 – 00:10

Day 10 Recap & Q&A

SQLite, the expense tracker, the `with` block, pandas bridge. Quick check.

00:10 – 00:30

Regex — Pattern Matching

`re.findall`, `re.search`, `re.sub`. Anchors, quantifiers, character classes, groups.

00:30 – 00:45

Reading PDFs with PyPDF2

Extract text from PDF files. Handle multi-page, handle failures.

00:45 – 00:55

Reading Word Docs

`python-docx` for `.docx` files. Extract paragraphs, tables, formatting.

00:55 – 01:10

Cheatsheet + Common Patterns

The 10 regex patterns you'll reuse. Email, phone, date, URL, and more.

01:10 – 01:30

Exercise: Resume Contact Extractor

Extract name, email, phone from a sample resume PDF. Combines regex + PyPDF2.

RECAP · 10 MIN

Yesterday you built with a real database. Today your app learns to read.

Quick check that Day 10's expense tracker runs. Today's exercise uses SQLite to store extracted contacts — so you'll need the database pattern fresh.

WHAT WE COVERED YESTERDAY

DAY 10

- **ALTER TABLE + DROP TABLE** — changing table structure
- **SQLite full pattern** — connect → CREATE → INSERT → commit
- **SQLite CRUD** — SELECT, INSERT, UPDATE, DELETE
- **SQLite in Streamlit** — the helper-function pattern
- **The `with` context manager** — auto-close connections

CHECK YOUR LAPTOP NOW

VERIFY

```
# You should have Day 10's expense tracker:
$ ls day10/
expense_sqlite.py
expenses.db

# Run it — should open in browser:
$ streamlit run expense_sqlite.py
# → Form works, expenses persist in SQLite
```

⚠ **IF THE APP DOESN'T RUN** — raise your hand. Today's exercise stores extracted contacts in SQLite, so you need the helper-function pattern working.

CONCEPT · WHY THIS MATTERS

4 of the 10 group project ideas need text extraction.

Resume matcher, plagiarism checker, smart notes summarizer, campus FAQ assistant — all start with extracting text from files. Today you learn the foundation.

GROUP PROJECTS THAT NEED THIS **PROJECT IDEAS**

1. **Resume vs Job Description Matcher**
   Upload a resume PDF, extract text, compare to a job description. Day 13's AI technique + today's PDF extraction.
2. **Plagiarism & Similarity Checker**
   Compare two or more documents. Extract text from each, compute similarity. Today's regex + Day 13's TF-IDF.
3. **Smart Notes Summarizer**
   Paste in lecture notes, get the key sentences. Text splitting + TF-IDF scoring. Regex for sentence boundaries.
4. **Campus FAQ Assistant**
   Match student questions to stored answers. Text similarity. Regex for cleaning input.

WHAT YOU LEARN TODAY **SESSION 11**

Regex
Find patterns in text. Emails, phone numbers, dates, URLs. Python's `re` module.

PDF extraction
Pull text out of PDF files using PyPDF2. The first step in resume matching.

Word doc extraction
Read `.docx` files using `python-docx`. For when the resume is a Word doc, not a PDF.

Text cleaning
Lowercase, strip whitespace, remove special chars. The boring 80% of text processing.

THE PATTERN **Every text-processing app: extract text (regex/PDF/Word) → clean it (lowercase, strip) → analyze it (similarity, frequency, AI).** Today is step 1 + 2. Day 13 is step 3.

Regex — find patterns in text without knowing the exact text.

Regular expressions are a mini-language for describing text patterns. “Find all email addresses” or “find all phone numbers” — regex makes this one line of code instead of 50.

WITHOUT REGEX · THE HARD WAY — 20 LINES

```
# Find all email addresses in a text
text = "Contact: aarav@example.com or priya@college.edu"

# Manual approach — split, check @, check domain...
words = text.split()
emails = []
for word in words:
    if "@" in word:
        # But what about punctuation? What about
        # aarav@example.com, (with a comma)?
        # What about .edu vs .com vs .co.in?
        # This gets complicated fast.
        cleaned = word.strip(",.!?;")
        if "." in cleaned.split("@")[-1]:
            emails.append(cleaned)
# → ['aarav@example.com', 'priya@college.edu']
# Works... barely. Misses edge cases.
```

⚠ **20 lines.** Brittle. Misses edge cases. Hard to maintain.

WITH REGEX · ONE LINE — 1 LINE

```
import re

text = "Contact: aarav@example.com or priya@college.edu"

# One line. Finds ALL email addresses.
emails = re.findall(r'\S+@\S+\.\S+', text)
# → ['aarav@example.com', 'priya@college.edu']

# What the pattern means:
# \S+  = one or more non-space chars (the username)
# @    = literal @
# \S+  = one or more non-space chars (the domain)
# \.   = literal dot
# \S+  = one or more non-space chars (the TLD)
```

✓ **1 line.** Handles edge cases. Industry standard.

KEY IDEA Regex is a pattern-matching language. You describe **WHAT** you want (“all emails”), not **HOW** to find it (“split on space, check for @”). Python’s `re` module runs the patterns.

Three functions. That's 90% of regex.

`re.findall` finds all matches. `re.search` finds the first match. `re.sub` replaces matches. Learn these three, you can do almost anything.

`re.findall(pattern, text)`

Find ALL matches. Returns a list of strings.

```
import
 re
text =
"Call 9876543210 or 9123456789"

phones = re.
findall
(
```

`re.search(pattern, text)`

Find the FIRST match. Returns a Match object (or `None`).

```
match = re.
search
(
r'\d{10}'
, text)
if
 match:
```

`re.sub(pattern, replacement, text)`

Replace matches with a new string. Returns the modified text.

```
# Mask all phone numbers

masked = re.
sub
(
r'\d{10}'
,
'XXXXXXXXXX'
# → 'Call XXXXXXXXXX or XXXXXXXXXX'
# Use sub to redact, clean, or transform text.
```

THE PATTERN — **findall** (get all), **search** (get first/check exists), **sub** (replace). Use raw strings (`r'...'`) for patterns so backslashes work correctly.

Two things that trip up every regex beginner.

Special characters need escaping. And quantifiers are greedy by default. Get these two right and 90% of your regex bugs disappear.

1 GOTCHA · ESCAPING

`.` → `\.` any char	`*` → `\*` 0 or more	`+` → `\+` 1 or more
`?` → `\?` 0 or 1	`^` → `\^` start	`$` → `\$` end
`[` → `\[` class	`(` → `\(` group	`\` → `\\` escape

```
# Find a literal dot (like in a URL)
text = "Visit example.com or test.org"
# ✗ Bad: r'.' matches EVERY character
# ✓ Good: r'\.' matches only the dot
dots = re.findall(r'\.', text)
# → ['.', '.']
```

Match a special char literally — put a `\` before it.

2 GOTCHA · GREEDY VS LAZY

`.*` GREEDY (DEFAULT) — AS MUCH AS POSSIBLE

```
text = "<b>Hello</b> <b>World</b>"
# Greedy: matches from first < to last >
greedy = re.findall(r'<b>.*</b>', text)
# → ['<b>Hello</b> <b>World</b>']
# ← ONE match, too much!
```

`.*?` LAZY (ADD ?) — AS LITTLE AS POSSIBLE

```
# Lazy: matches each <b>...</b> separately
lazy = re.findall(r'<b>.*?</b>', text)
# → ['<b>Hello</b>', '<b>World</b>']
# ← TWO matches, correct!
```

Add `?` after any quantifier to make it lazy: `*?` `+?` `??` `{n,m}?`

THE RULE **Escape** special chars with `\`. Make quantifiers **lazy** with `?`. These two habits prevent 90% of regex bugs. When in doubt, test on regex101.com.

Three flags that change how your pattern behaves.

Flags modify regex behavior. Case-insensitive matching, multi-line mode, and dot-matches-newline. Pass them as the third argument or use inline `(?i)` syntax.

`re.IGNORECASE` or re.I

Case-insensitive matching. `A` matches `a` and `A`. The most used flag.

```
import
 re
text =
"Python is Great. python is fun. PYTHON rocks."
# Without flag: only matches 'Python' (exact case)

matches = re.
findall
```

`re.MULTILINE` or re.M

Makes `^` and `$` match the start/end of each LINE, not just the whole string.

```
text =
"line one\nline two\nline three"
# Without flag: ^ only matches start of entire string
# With MULTILINE: ^ matches start of each line

matches = re.
findall
(
```

`re.DOTALL` or re.S

Makes `.` match newlines too. By default, `.` matches everything EXCEPT newline.

```
text =
"Hello\nWorld"
# Without DOTALL: . doesn't match \n
# r'.+' → ['Hello', 'World']  ← two matches
# With DOTALL: . matches everything including \n

matches = re.
findall
```

```
# → [ 'Hello\nWorld' ]  ← one match, includes newline
```

THE PATTERN

Pass flags as the **3rd argument**: `re.findall(pattern, text, re.IGNORECASE)`. Combine with `|`: `re.I | re.M`. Or use inline: `r'(?i)pattern'` for IGNORECASE, `r'(?m)pattern'` for MULTILINE.

Four building blocks. Every regex pattern is a combination of these.

Anchors (where to start/end), quantifiers (how many), character classes (what kind of char), groups (sub-patterns). Learn these four, you can read any regex.

ANCHORS · WHERE

SYM	MEANING	EXAMPLE
`^`	Start of string	`^Hello matches 'Hello' at start`
`$`	End of string	`world$ matches 'world' at end`
`\b`	Word boundary	`\bcat\b matches 'cat' not 'catch'`

QUANTIFIERS · HOW MANY

SYM	MEANING	EXAMPLE
`*`	0 or more	`\d* matches '', '5', '123'`
`+`	1 or more	`\d+ matches '5', '123' (not '')`
`?`	0 or 1	`colou?r matches 'color', 'colour'`
`{n}`	Exactly n	`\d{10} matches '9876543210'`
`{n,m}`	Between n and m	`\d{2,4} matches '12', '123', '1234'`

CHARACTER CLASSES · WHAT KIND

SYM	MEANING	EXAMPLE
`.`	Any char except newline	`a.c matches 'abc', 'axc'`
`\d`	Digit (0-9)	`\d+ matches '123'`
`\w`	Word char (a-z, A-Z, 0-9, _)	`\w+ matches 'hello_123'`
`\s`	Whitespace	`\s+ matches ' '`
`[abc]`	Any of a, b, c	`[aeiou] matches 'a'`
`[^abc]`	NOT a, b, c	`[^0-9] matches non-digits`

GROUPS · SUB-PATTERNS

SYM	MEANING	EXAMPLE
`()`	Capture group	`(\d{3})-(\d{4}) matches '123-4567'`
`|`	OR	`cat|dog matches 'cat' or 'dog'`
`(?:...)`	Non-capture group	`(?:abc)+ matches 'abcabc'`

THE CHEAT Start with `\d` (digits), `\w` (words), `+` (one or more), and `[]` (choices). That's 80% of real-world regex. The rest is combining them.

Three patterns you'll actually use.

These aren't toy examples — they're the exact patterns from the exercise, the group project, and real job interviews.

01 EXTRACT ALL EMAIL ADDRESSES

WHEN

You have a resume or a contact list. Find every email in it.

```
import re
text = "Email me at aarav@gmail.com or priya@yahoo.co.in"
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', text)
# → ['aarav@gmail.com', 'priya@yahoo.co.in']
# Breakdown:
# [\w.+-]+  = username (letters, digits, dots, +, -)
# @         = literal @
# [\w-]+    = domain name
# \.        = literal dot
# [\w.]+    = TLD (.com, .co.in, .edu)
```

02 EXTRACT INDIAN PHONE NUMBERS

WHEN

Find all 10-digit phone numbers in a text. Handles +91 prefix and spaces/dashes.

```
text = "Call +91 98765 43210 or 91234-56789"
phones = re.findall(r'\+?\d[\d\s-]{8,}\d', text)
# → ['+91 98765 43210', '91234-56789']
# Breakdown:
# \+?       = optional + (for +91)
# \d        = starts with a digit
# [\d\s-]{8,} = 8+ digits, spaces, or dashes
# \d        = ends with a digit
```

03 EXTRACT DATES (YYYY-MM-DD)

WHEN

Find all dates in a log file or document.

```
text = "Joined 2023-08-15, left 2024-01-20"
dates = re.findall(r'\d{4}-\d{2}-\d{2}', text)
# → ['2023-08-15', '2024-01-20']
# Breakdown:
# \d{4} = exactly 4 digits (year)
# -     = literal dash
# \d{2} = exactly 2 digits (month)
# -     = literal dash
# \d{2} = exactly 2 digits (day)
```

re.sub — find and replace with patterns.

The third **re** function. Used less than **findall**, but essential for cleaning text — removing punctuation, normalizing whitespace, masking sensitive data.

sub_demo.py — Python · re

```python
import re

text = "Hello,  World!  This   has   extra   spaces."

# 1. Replace multiple spaces with single space
cleaned = re.sub(r'\s+', ' ', text)
# → 'Hello, World! This has extra spaces.'

# 2. Remove all punctuation
no_punct = re.sub(r'[^\w\s]', '', text)
# → 'Hello  World  This   has   extra   spaces'

# 3. Mask phone numbers (redact sensitive data)
text2 = "Call 9876543210 for details"
masked = re.sub(r'\d{10}', 'XXXXXXXXXX', text2)
# → 'Call XXXXXXXXXX for details'

# 4. Replace with a capture group (\1)
text3 = "John, Doe"
swapped = re.sub(r'(\w+), (\w+)', r'\2 \1', text3)
# → 'Doe John'  (last name, first name → first last)
```

WHEN TO USE SUB

4 use cases

Clean text
Normalize whitespace, remove punctuation, lowercase. The boring 80% of text processing.

Mask sensitive data
Replace phone numbers, emails, Aadhaar numbers with Xs before logging or displaying.

Transform format
Swap date formats (YYYY-MM-DD → DD/MM/YYYY), reorder names (Last, First → First Last).

Normalize quotes
Replace smart quotes (" ") with straight quotes (") for consistency.

THE PATTERN extract (**findall**) → clean (**sub**) → analyze. **re.sub** is the cleaning step. You'll use it to prepare text before the AI exercises on Day 13.

02 / 03 CHAPTER

Text doesn't live in strings. It *lives in files.*

Resumes are PDFs. Assignments are Word docs. Lecture notes are PDFs. Today we learn to pull text out of these formats — the first step of every text-processing app.

PDFs with PyPDF2 · `pip install`

Word docs with python-docx · `pip install`

PyPDF2 — pull text out of PDF files.

PDFs are designed for display, not data extraction. PyPDF2 reads the text layer — when it exists. Not all PDFs have extractable text (scanned PDFs are images, not text).

pdf_demo.py — PyPDF2 · 5 steps

```python
# Install first: pip install PyPDF2
from PyPDF2 import PdfReader

# 1. Open the PDF
reader = PdfReader("resume.pdf")

# 2. How many pages?
print(f"Pages: {len(reader.pages)}")
# → Pages: 2

# 3. Extract text from each page
full_text = ""
for page in reader.pages:
    full_text += page.extract_text() + "\n"

print(full_text[:500])  # first 500 chars
# → "John Doe\nSoftware Engineer\n..."

# 4. Extract from a specific page
page1_text = reader.pages[0].extract_text()

# 5. Now use regex on the extracted text
import re
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', full_text)
print(emails)
# → ['john@example.com']
```

GOTCHAS + INSTALL

5 things to know

Install

`pip install PyPDF2`. Not built in — you need to install it. Run this in your `.venv`.

Scanned PDFs

If the PDF is a scan (image, not text), PyPDF2 returns empty string. You'd need OCR (optical character recognition) — too advanced for today.

Formatting loss

PDFs don't have "paragraphs" or "tables" in the data sense. Text comes out as a flat string. You'll need regex to find structure.

Multi-column layouts

Two-column PDFs (like academic papers) may extract in the wrong reading order. Text comes out jumbled.

Cross-ref: Day 9

Store extracted text in SQLite (Day 10's pattern) for later querying.

THE PATTERN — Open PDF → loop pages → `extract_text()` → concatenate → use regex. This is the exact pipeline for the resume matcher on Day 13.

python-docx — read .docx files. Better than PDFs.

Word docs have actual structure — paragraphs, tables, styles. python-docx extracts them cleanly. If you have a choice, ask for .docx not .pdf.

docx_demo.py — python-docx · 5 steps

```python
# Install first: pip install python-docx
from docx import Document

# 1. Open the .docx file
doc = Document("report.docx")

# 2. Extract all paragraphs
full_text = ""
for para in doc.paragraphs:
    full_text += para.text + "\n"

print(full_text[:500])
# → "Monthly Report\nAugust 2026\n..."

# 3. Extract tables (PDFs can't do this!)
for table in doc.tables:
    for row in table.rows:
        cells = [cell.text for cell in row.cells]
        print(cells)
# → ['Name', 'Marks'], ['Aarav', '85'], ...

# 4. Get paragraph styles (heading, body, etc.)
for para in doc.paragraphs:
    if para.style.name == "Heading 1":
        print(f"HEADING: {para.text}")

# 5. Use regex on the extracted text
import re
emails = re.findall(r'[\w.+-]+@[\w-]+\.[\w.]+', full_text)
```

PDF VS WORD DOC

7 features

FEATURE	PDF (PyPDF2)	Word (python-docx)
Text extraction	Flat string	Paragraphs + tables
Tables	✕ Lost	✓ **Preserved**
Headings	✕ Lost	✓ **Style names**
Images	✕ Ignored	✓ **Accessible**
Scanned docs	✕ Empty	N/A
Install	`pip install PyPDF2`	`pip install python-docx`
Best for	Resumes, reports, papers	Structured documents

CHEATSHEET · 10 PATTERNS

Bookmark this. These 10 patterns cover 90% of real-world regex.

Copy-paste, adapt, use. Each pattern has been tested against Indian formats (phone numbers, dates, postal codes).

EMAIL 01

```
[\w.+-]+@[\w-]+\.[\w.]+
```

MATCHES `aarav@gmail.com`, `priya@yahoo.co.in`

```
re.findall(pattern, text)
```

PHONE (INDIAN) 02

```
\+?\d[\d\s-]{8,}\d
```

MATCHES `9876543210`, `+91 98765 43210`, `91234-56789`

```
re.findall(pattern, text)
```

DATE (YYYY-MM-DD) 03

```
\d{4}-\d{2}-\d{2}
```

MATCHES `2026-08-07`, `2023-12-25`

```
re.findall(pattern, text)
```

DATE (DD/MM/YYYY) 04

```
\d{2}/\d{2}/\d{4}
```

MATCHES `07/08/2026`, `25/12/2023`

```
re.findall(pattern, text)
```

URL 05

```
https?://[\w.-]+
```

MATCHES `http://example.com`, `https://www.google.com`

```
re.findall(pattern, text)
```

IP ADDRESS 06

```
\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}
```

MATCHES `192.168.1.1`, `10.0.0.1`

```
re.findall(pattern, text)
```

POSTAL CODE (INDIA) 07

```
\d{6}
```

MATCHES `143001`, `110001`

```
re.findall(pattern, text)
```

NUMBER (WITH DECIMALS) 08

```
\d+\.?\d*
```

MATCHES `42`, `3.14`, `0.5`

```
re.findall(pattern, text)
```

HASHTAG 09

```
#\w+
```

MATCHES `#Python`, `#DataScience`

```
re.findall(pattern, text)
```

WORD (LETTERS ONLY) 10

```
[a-zA-Z]+
```

MATCHES `hello`, `Python`

```
re.findall(pattern, text)
```

TEST FIRST — Always test your regex on real text before trusting it. Use `regex101.com` or just `print(re.findall(pattern, your_text))`. Regex is powerful but not magic — edge cases exist.

03 / 03

Build a resume contact extractor.

Extract name, email, and phone from a sample resume PDF. Combines everything from today — PyPDF2 + regex — and sets up Day 13's AI resume matcher.

01 **Brief** (5 MIN) 02 **Build** (15 MIN) 03 **Extend** (5 MIN)

EXERCISE · BRIEF · 5 MIN

Extract contacts from a resume PDF. Name, email, phone.

A real-world task. Upload a resume, pull out the contact info. This is the same pipeline Day 13's resume matcher starts with — extract text, find patterns.

THE BRIEF

Task

Build a Python script called `resume_extractor.py` that reads a sample resume PDF (provided), extracts the text, and uses regex to find: the email address, the phone number, and (bonus) the person's name.

REQUIREMENTS

- ✓ Use PyPDF2 to extract text from `sample_resume.pdf` (provided in shared folder)
- ✓ Use `re.findall` to find the email address
- ✓ Use `re.findall` to find the phone number
- ✓ Print the results clearly: `'Email: ...'`, `'Phone: ...'`
- ✓ Bonus: Extract the name (hint: it's usually the first line of the resume)

BY THE END

Every student has extracted contacts from a PDF using regex. The Day 13 pipeline.

STARTING SCAFFOLD

resume_extractor.py

```python
# resume_extractor.py — starting scaffold
import re
from PyPDF2 import PdfReader

# 1. Read the PDF
reader = PdfReader("sample_resume.pdf")
full_text = ""
for page in reader.pages:
    full_text += page.extract_text() + "\n"

print("=== EXTRACTED TEXT (first 300 chars) ===")
print(full_text[:300])

# 2. Find the email
# TODO: email_pattern = r'...'
# TODO: emails = re.findall(email_pattern, full_text)
# TODO: print(f"Email: {emails[0] if emails else 'Not found'}")

# 3. Find the phone number
# TODO: phone_pattern = r'...'
# TODO: phones = re.findall(phone_pattern, full_text)
# TODO: print(f"Phone: {phones[0] if phones else 'Not found'}")

# 4. BONUS: Find the name (first non-empty line)
# TODO: lines = full_text.strip().split("\n")
```

RUN IT — `python resume_extractor.py`. The PDF is in the shared folder. If regex doesn't find the email, print the full text and look at it — the pattern might need adjusting.

EXERCISE · SOLUTION WALKTHROUGH

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

resume_extractor.py

PYTHON · PYPDF2 · RE · SQLITE3

```python
import re
from PyPDF2 import PdfReader

1 reader = PdfReader("sample_resume.pdf")
  full_text = ""
  for page in reader.pages:
      full_text += page.extract_text() + "\n"

2 email_pattern = r"[\w.+-]+@[\w-]+\.[\w.]+"
  emails = re.findall(email_pattern, full_text)
  print(f"Email: {emails[0] if emails else 'Not found'}")

3 phone_pattern = r"\+?\d[\d\s-]{8,}\d"
  phones = re.findall(phone_pattern, full_text)
  print(f"Phone: {phones[0] if phones else 'Not found'}")

4 lines = [line.strip() for line in full_text.split("\n") if line.strip()]
  name = lines[0] if lines else "Unknown"
  print(f"Name: {name}")

5 # Store in SQLite (Day 10 pattern)
  import sqlite3
  conn = sqlite3.connect("contacts.db")
  conn.execute("""
      CREATE TABLE IF NOT EXISTS contacts (
          name TEXT, email TEXT, phone TEXT
      )
  """)
  conn.execute(
      "INSERT INTO contacts VALUES (?, ?, ?)",
```

LINE BY LINE

1 Read the PDF

`PdfReader` opens the file. Loop pages, `extract_text()` per page, concatenate. Day 9's pattern.

2 Find email

The email pattern from the cheatsheet (slide 13). `findall` returns a list — take `[0]`.

3 Find phone

Indian phone pattern. Handles `+91`, spaces, dashes. Same pattern from slide 8.

4 Find name

The name is usually the first non-empty line. Split on newlines, filter empty, take `[0]`.

5 Store in SQLite

Bonus: persist the extracted contact. Day 10's helper pattern. The group project will do this.

THE PIPELINE — Extract (PDF) → Find (regex) → Store (SQLite). This is the exact pipeline for Day 13's resume matcher — plus a TF-IDF similarity step.

EXERCISE · EXTEND + WHAT'S NEXT

Finished early? Extract more fields. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Day 12 — Calling Real APIs.

EXTENSION IDEAS

For Fast Students

1 Extract skills

After the name/email/phone, find a "Skills" section in the resume. Look for keywords like Python, Java, SQL, HTML. Hint: find the line with "Skills", then grab the next few lines.

2 Handle Word docs too

Add `python-docx` support. If the file is `.pdf` use `PyPDF2`, if `.docx` use `python-docx`. Detect by file extension.

3 Build a Streamlit UI

Wrap the script in a Streamlit app. `st.file_uploader` for the resume, `st.write` for the results. The group project pattern.

WHAT'S NEXT · DAY 12

Calling Real APIs

Live data from the internet

Weather, currency rates, news. Python's `requests` library.

JSON responses

APIs return JSON. You learned `json.load` on Day 9 — now use it on live data.

Real apps use APIs

Every app that shows weather, stock prices, or news is calling an API.

TOMORROW — Your apps stop being self-contained. They pull live data from the internet. The bridge to real-world software.

IF YOU'RE STUCK — Print the extracted text. Look at it. Adjust your regex. Regex is iterative — first try rarely catches everything.

Q&A · CLOSING · 5 MIN

Your apps can now read.

Tomorrow they learn to reach out — live data from the internet. The day your apps stop being self-contained.

OPEN THE FLOOR

Ask Now

- ✓ "Which regex pattern felt most useful — email, phone, or date?"
- ✓ "Did PDF extraction work on the sample resume, or was the text jumbled?"
- ✓ "Anything from the exercise that didn't work?"

BETWEEN SESSIONS

Get Unstuck

- ✓ **Email** · `hello@zlaark.com`
  For regex questions, paste your pattern + the text + what it should match.
- ✓ **Web** · `www.zlaark.com`
  Program updates and resources.
- ✓ **Practice — try** `regex101.com`
  with your own patterns.

WHAT'S NEXT · DAYS 12–13

Phase 3 Continues

- ✓ Day 12 — Calling Real APIs. Live data from the internet.
- ✓ Day 13 — A First Taste of AI. TF-IDF, text similarity.
- ✓ Day 13 uses today's skills — resume matcher = PDF extraction + TF-IDF.

Zlaark

See you on Day 12.